

# Mini Hiring Pipeline — Architecture Summary

**GitHub Repository:** [https://github.com/Deekshithpatkar/Hiring\\_Pipeline](https://github.com/Deekshithpatkar/Hiring_Pipeline)

**Candidate:** Deekshith Patkar | **Role:** Technical Take-Home Assignment

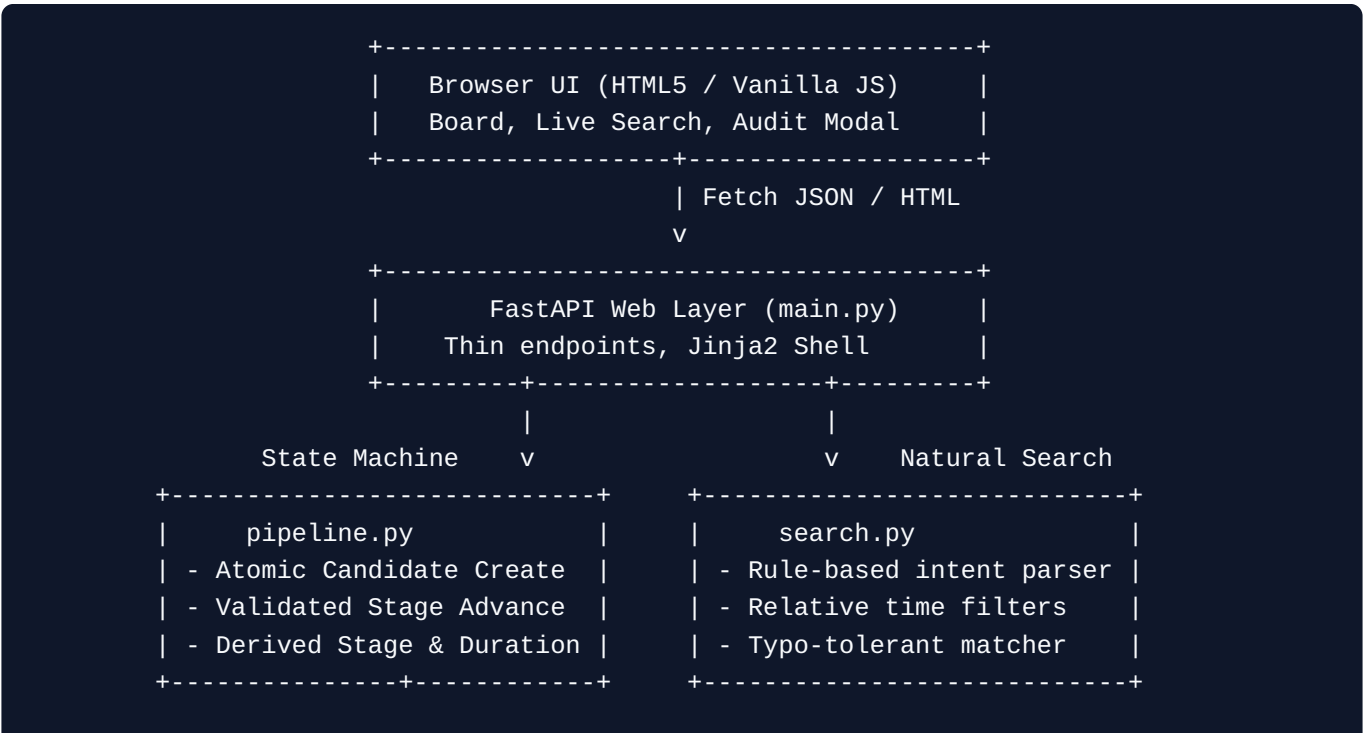
**Stack:** Python 3.14, FastAPI, SQLAlchemy ORM, PostgreSQL 18, Jinja2, Vanilla JS / CSS

## 1. System Overview & Core Principles

The Mini Hiring Pipeline is built to manage candidates for a job through a linear stage sequence with three non-negotiable architectural guarantees:

- **Single Source of Truth:** There is no mutable `current_stage` column anywhere. A candidate's current stage is always dynamically derived as the `to_stage` of their latest recorded `StageEvent` . State drift is impossible by design.
- **Database-Level Immutability:** The audit trail is strictly append-only. A PostgreSQL `BEFORE UPDATE OR DELETE` trigger raises an exception directly inside the database engine if any modification or deletion is attempted.
- **Deterministic, Explainable Logic:** State machine validation and search query parsing are written in plain, transparent Python without relying on external NLP libraries or non-deterministic LLM calls.

## 2. Architecture Diagram



```
      |  
      v SQLAlchemy ORM  
+-----+  
|               PostgreSQL Database               |  
| 1. candidates (id, name, email, created_at)      |  
| 2. stage_events (id, candidate_id, from, to, timestamp) |  
| * Immutable: Enforced by Postgres BEFORE trigger |  
| * Unique: (candidate_id, to_stage) constraint    |  
+-----+
```

### 3. Data Model & Integrity Guarantees

The database schema consists strictly of two tables:

- `candidates` : Stores the candidate identity ( `id`: UUID , `name`: TEXT , `email`: TEXT , `created_at`: TIMESTAMPTZ ).
- `stage_events` : The append-only audit trail ( `id`: UUID , `candidate_id`: UUID FK , `from_stage`: TEXT NULL , `to_stage`: TEXT , `timestamp`: TIMESTAMPTZ ).

#### PostgreSQL Immutability Trigger

```
CREATE OR REPLACE FUNCTION prevent_stage_events_modification()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'stage_events audit log is immutable and cannot be updated or deleted';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_stage_events_immutable
BEFORE UPDATE OR DELETE ON stage_events
FOR EACH ROW
EXECUTE FUNCTION prevent_stage_events_modification();
```

#### Preventing Duplicate Stage Events

A table-level constraint `uq_candidate_to_stage` (`candidate_id`, `to_stage`) guarantees at the database level that no candidate can ever re-enter a stage they have previously occupied.

### 4. State Machine Implementation

The pipeline transitions candidates along a strict, forward-only path:

**Applied → Screening → Interview → Offer → Hired**

- **Atomic Creation:** Creating a candidate writes the `Candidate` row and the initial `from_stage: NULL, to_stage: 'Applied'` event in one database transaction with automatic rollback on error.
- **No Skipping or Reversals:** Candidates can advance only one stage forward at a time. Backward moves (e.g. `Interview → Screening`) or skipped stages (e.g. `Applied → Interview`) are rejected with clear, explanatory error strings.
- **Rejection Handling:** `Rejected` is reachable from any stage before `Hired`. Both `Hired` and `Rejected` are terminal states.

- **Inline Error Feedback:** All validation errors return structured HTTP 400 responses with exact descriptions, enabling immediate inline feedback on candidate cards without confirmation dialogs.

## 5. Search Engine & Ranking

---

The search engine uses a rule-based parser in pure Python (no LLM, no external NLP dependencies):

- **Fuzzy Typo-Tolerant Matching:** Employs `difflib.SequenceMatcher` with token analysis at a 0.75 threshold (e.g. "sharam" → "Sharma" ). Exact matches rank above fuzzy matches.
- **Relative Time & Transition Filters:** Resolves phrases such as "stuck in Screening for more than a week" (> 7 days) and "moved to Interview since Monday" into UTC timestamp comparisons.
- **Reached-but-not-current:** Identifies queries like "reached offer but didn't get hired" by verifying historical `offer` events while ensuring the current stage is not `Hired` .
- **Negation:** Parses "everyone except rejected" into an exclusion filter ( `current_stage != 'Rejected'` ).
- **Diagnostic Guidance:** Unrecognized queries (e.g. "who's in onboarding" ) return explicit explanations of what was invalid rather than silent empty lists.

## 6. Known Limitations & What I'd Do With More Time

---

Two gaps I found through my own testing, rather than ones I'm aware of only in theory:

- **Timestamp ordering in `advance_stage()`** : The function accepts an optional explicit `event_time` parameter, but does not currently validate that it falls after the candidate's most recent recorded event. The live API does not expose this parameter, so it cannot be triggered through the UI today — but calling the function directly with a backdated timestamp can cause the derived "current stage" to disagree with the candidate's own event history. With more time, I would add a check that rejects any `event_time` at or before the latest existing event for that candidate.
- **Search filler-word stripping is not exhaustive**: After removing recognized stage names, time phrases, and negation words, the remaining text is treated as a name query. This remainder is not always fully cleaned of conversational words (e.g. "show me candidates named Priya" currently leaves "named Priya" rather than "Priya"). Results for the assignment's example queries are unaffected, since the fuzzy matcher is tolerant enough to still find the right candidate, but a dedicated stopword list would make this more robust for less predictable phrasing.

Additional extensions I'd consider for a more general-purpose version of this tool:

- **Multi-round stages**: Currently each stage (e.g. Interview) can only be entered once per candidate. A real pipeline often has multiple rounds within one stage; I'd add a `round_label` column to `stage_events` and loosen the uniqueness constraint to (candidate, stage, round) to support this without changing the overall model.
- **Multi-job / role scoping**: The brief scopes this to a single job, so no `jobs` entity exists. A multi-role version would need candidates linked to a specific job posting.
- **pg\_trgm -based search**: For a much larger candidate set, moving fuzzy name matching into Postgres itself via the `pg_trgm` extension would scale better than in-application scoring.

## 7. Verification

---

Rather than assert these guarantees, I tested them directly:

- **Database-level immutability**: Ran raw SQL `UPDATE` and `DELETE` statements against `stage_events` directly in `psql`, outside the application layer entirely. Both were rejected by the Postgres trigger with the expected exception.
- **State machine rules**: Verified every transition rule — the full valid path, stage-skipping, backward moves, duplicate transitions, and moves attempted from a terminal state — via direct HTTP requests to the API (Postman/curl), independent of the UI, to confirm the guarantee holds even when the frontend is bypassed entirely.

- **Timestamp ordering gap:** Reproduced the limitation described in Section 6 by calling `advance_stage()` directly with a backdated `event_time` and confirming the resulting mismatch between the candidate's derived current stage and their recorded history.
- **Search examples:** Ran the assignment's example queries end to end against a live instance — typo-tolerant name search, current-stage filters, combined and negated filters, and an unrecognized-stage query — and confirmed each returned correct, explained results.